



Programovacie jazyky

Haskell – úvod do jazyka (funkcie, chvostová rekurzia)

RNDr. Richard Ostertág, PhD.

(ostertag@dcs.fmph.uniba.sk)

28. mája 2017

Katedra informatiky, Fakulta matematiky, fyziky a informatiky
Univerzita Komenského, Bratislava

Rôzne spôsoby definovania funkcií

Faktoriál – bez rekurzie, použitím iných funkcií

```
1  -- voliteľne možno zadať anotáciu typu, obvykle pre
2  --      * zúženie polymorfizmu
3  --      * optimalizácie pre zvýšenie výkonu
4  --      * zvýšenie čitateľnosti kódu
5  factorial :: (Num a, Enum a) => a -> a
6  factorial :: (Integral a) => a -> a
7  factorial :: Integer -> Integer
8  factorial :: Int -> Int
9
10 -- Použitím funkcie product a aritmetickej postupnosti (zoznamu)
11 factorial n = product [1..n]
12
13 -- rozpíšeme [1..n] (cez typovú triedu Enum)
14 factorial n = product (enumFromTo 1 n)
15
16 -- point-free style :-)
17 factorial = product . enumFromTo 1
```

Faktoriál – rekurzia, bez porovnávania so vzorom

```
1 factorial :: (Ord a, Num a) =>
2     a -> a
3
4 factorial n =
5     if n > 0
6         then n * factorial (n - 1)
7         else 1
```

Faktoriál – rekurzia, porovnávanie so vzorom

```
1 -- Porovnávanie so vzorom (pattern matching)
2 factorial :: (Eq a, Num a) => a -> a
3 factorial 0 = 1
4 factorial 1 = 1
5 factorial 9 = 362880
6 factorial n = n * factorial (n - 1)
7
8 -- Porovnávanie so vzorom s podmienkou (guards)
9 factorial :: (Ord a, Num a) => a -> a
10 factorial n | n < 2      = 1
11              | otherwise = n * factorial (n - 1)
```

`otherwise` nie je kľúčové slovo, ale iba konštanta:

```
1 otherwise :: Bool
2 otherwise = True
```

Neúplné porovnávanie so vzorom (chyba)

```
1 neg :: Bool -> Bool
2 neg True = False
```

- warning: [-Wincomplete-patterns]
 - Pattern match(es) are **non-exhaustive**
 - In an equation for 'neg': **Patterns not matched: False**

Čo sa stane, ak zavoláme **neg False**? Runtime error. :-(

```
1 *Main> neg False
2 *** Exception: Non-exhaustive patterns in function neg
```

Neignorujeme dobre myslené varovania. :-) Oprava:

```
1 neg True  = False
2 neg False = True
```

Neúplné porovnávanie so vzorom (čiasočná funkcia)

```
1 prvy :: [a] -> a
2 prvy (x:xs) = x
```

- warning: [-Wincomplete-patterns]
 - Pattern match(es) are **non-exhaustive**
 - In an equation for 'prvy': **Patterns not matched: []**
- warning: [-Wunused-matches]
 - Defined but not used: 'xs'

Riešenie (explicitne čiasočná funkcia):

```
1 prvy (x:_xs) = x
2 prvy [] = undefined -- *** Exception: Prelude.undefined
3 prvy [] = error "Toto naozaj nie je totálna funkcia."
```

Lepšie riešenie (totálna funkcia):

```
1 prvy :: [a] -> Maybe a
2 prvy [] = Nothing
3 prvy (x:_xs) = Just x
```

Neúplné porovnávanie so vzorom (zložitejší príklad)

```
1 troj :: [a] -> Bool
2 troj [] = True
3 troj [_] = False
4 troj (_:_:xs) = troj xs
```

- warning: [-Wincomplete-patterns]
 - Pattern match(es) are **non-exhaustive**
 - In an equation for ‘troj’: **Patterns not matched: [_, _]**

```
1 *Main> troj [] -- True
2 *Main> troj [1] -- False
3 *Main> troj [1,2] -- False, ale:
4 *** Exception: Non-exhaustive patterns in function troj
5 *Main> troj [1,2,3] -- True
6 *Main> troj [1,2,3,4] -- False
7 *Main> troj [1,2,3,4,5] -- False, ale:
8 *** Exception: Non-exhaustive patterns in function troj
9 *Main> troj [1,2,3,4,5,6] -- True
```


Neúplné porovnávanie so vzorom (zložitejší príklad)

Správne riešenie:

```
1 troj :: [a] -> Bool
2 troj [] = True
3 troj (_:_:_:xs) = troj xs
4 troj _ = False
```

Pozor na všetko zachytávajúci vzor „_“. Dobrý sluha, zlý pán.

Vzory s podmienkami je dobré ukončovať s `otherwise`, inak:

```
1 f x | x < 10 = "malo"
2     | x == 10 = "akurat"
3     | x > 10 = "veľa"
```

- warning: [-Wincomplete-patterns]
 - Pattern match(es) are **non-exhaustive**
 - In an equation for 'f': **Patterns not matched: _**

case výraz

```
1 case výraz_pre_porovnanie of
2   prvý_vzor -> výraz_pre_prvý_výsledok
3   ...       -> ...
4   ďalší_vzor -> výraz_pre_ďalší_výsledok
5   _         -> výraz_pre_výsledok_v_inom_prípade
```

- Porovnávanie:
 - vykonáva sa v poradí zhora nadol
 - zastaví sa pri prvej zhode
 - ak sa nenájde zhoda, tak skončí výnimkou

Definícia funkcie a **case**

Definícia funkcie pomocou porovnávania so vzorom:

```
1 f p11 ... p1k = e1
2 ...
3 f pn1 ... pnk = en
```

(pričom $p_{ij} = p_{i,j}$ je vzor a $en = e_n$ je výraz) je sémanticky ekvivalentná s definíciou pomocou **case**:

```
1 f x1 x2 ... xk =
2   case (x1, ..., xk) of
3     (p11, ..., p1k) -> e1
4     ...
5     (pn1, ..., pnk) -> en
```

Porovnávanie vzorov a sémantika

`take1` a `take2` sa líšia iba v poradí prvých dvoch rovností:

```
1 take1 _ [] = []  
2 take1 0 _ = []  
3 take1 n (x:xs) = x : take1 (n-1) xs
```

```
1 take2 0 _ = []  
2 take2 _ [] = []  
3 take2 n (x:xs) = x : take2 (n-1) xs
```

Definujú rovnakú funkciu až na správanie sa pre $\perp$:

- `take1` $0 \perp \rightsquigarrow \perp$
- `take2` $0 \perp \rightsquigarrow []$
 - funkcia `take2` je definovanejšia pre 2. argument
- `take2` $\perp [] \rightsquigarrow \perp$
- `take1` $\perp [] \rightsquigarrow []$
 - funkcia `take1` je definovanejšia pre 1. argument

Porovnávanie so vzorom (pattern matching)

_ (wildcard, anonymná premenná)

- `siadmaZdeviatich` `[_,_,_,_,_,_,_x,_,_]` = `x`
- opakovaný výskyt `_` nevadí
 - zakaždým ide o inú anonymnú premennú
- opakovanie neanonymnej premennej `f` `[x,x]` = `x`
 - vedie k chybe: `Conflicting definitions for 'x'`
 - ak niečo také treba, riešením sú podmienky:

```
1 f :: Eq a => [a] -> Maybe a
2 f [x,y] | x==y      = Just x
3         | otherwise = Nothing
```

@ (as-pattern, aliasing)

- `rastie` `(x:xs@(h:t))` = `x < h && rastie xs`
- hodí sa, ak chceme mať prístup k
 - celému výrazu (`xs`),
 - ako aj k jeho podčasti (`h` a `t`),
 - bez toho, aby sme ho po rozoberaní hneď následne skladali

let-výraz

Umožňuje definovať spoločné podvýrazy pre použitie v inom výraze:

```
1 let {- väzby -} in {- výraz -}
```

Let-výraz môže byť použitý všade tam, kde sa očakáva výraz.

```
1 povrchValca :: Floating a => a -> a -> a
2 povrchValca r v =
3     let obvodKruhu    = 2 * pi * r
4         obsahPlasta  = obvodKruhu * v
5         obsahPodstavy = pi * r ^ 2
6     in obsahPodstavy + obsahPlasta + obsahPodstavy
```

- Z použitia konštanty **pi** kompilátor odvodil, že typ **a** musí „implementovať“ typovú triedu **Floating** (inak by stačil **Num**).
- Ešte existuje let-príkaz (povieme si o ňom pri do-zápise).

where

Umožňuje definovať spoločné podvýrazy pre viacero podmienok porovnávania so vzorom:

```
1 bmiTell :: (RealFloat a) =>
2   a -> a -> String
3 bmiTell vyska_m hmotnost_kg
4   | bmi <= podvyziva = "anorexia?"
5   | bmi <= normal    = "ideal :-)"
6   | bmi <= nadvaha    = "nadvaha"
7   | bmi <= obezita    = "obezita"
8   | otherwise        = "velryba?"
9   where
10      bmi = hmotnost_kg /
11            vyska_m ^ 2
12      podvyziva = 18.5
13      normal    = 25.0
14      nadvaha    = 30.0
15      obezita    = 40.0
```

Rekurzia

Cyklus nahrádza rekurzia

```
1 lengthR :: [a] -> Int
2 lengthR [] = 0
3 lengthR (_x:xs) = 1 + lengthR xs
```

Takáto implementácia vyžaduje zásobník veľkosti $O(n)$.

Chvostové volania a rekurzia

Definícia

Hovoríme, že volanie funkcie g vo funkcii f je **chvostové volanie** (tail call) práve vtedy, ak toto volanie je posledným krokom vo výpočte f a výsledok volania funkcie g je teda priamo výsledkom funkcie f .

Príklady

- chvostové volania (g vo funkcii f):
 - $f\ x = g\ (x - 1)$
 - $f\ x = \text{if}\ (h\ x)\ \text{then}\ g\ (x - 1)\ \text{else}\ 0$
- nechvostové volania:
 - $f\ x = h\ (g\ (x - 1))$
 - $f\ x = (g\ (x - 1)) + 1$

Definícia

Rekurzívna funkcia f využíva **chvostovú rekurziu** (tail recursion) práve vtedy, ak všetky rekurzívne volania f sú chvostové volania.

Chvostová rekúzia

```
1 patri :: Eq a => a -> [a] -> Bool
2 patri _ [] = False
3 patri e (x:xs)
4     | e == x = True
5     | otherwise = patri e xs
```

Chvostová rekúzia umožňuje kompilátoru znovupoužiť aktívny záznam na zásobníku a tak stačí zásobník veľkosti $O(1)$.

Chvostová rekúzia s akumulátorom

Konverzia nechvostovej rekúzie na chvostovú rekúziu často vyžaduje použitie akumulátora.

```
1 length' :: [a] -> Int -> Int
2 length' [] a = a
3 length' (_:xs) a = length' xs (a + 1)
4
5 lengthTR :: [a] -> Int
6 lengthTR x = length' x 0
```

Pomocná funkcia `length'` sa da skryť pomocou **where**:

```
1 lengthTR :: [a] -> Int
2 lengthTR x = length' x 0
3     where length' :: [a] -> Int -> Int
4           length' [] a = a
5           length' (_:xs) a = length' xs (a + 1)
```

Chvostová rekurzia pomocou CPS

Každú rekurzívnu funkciu možno prepísať na chvostovú rekurziu aj použitím CPS (continuation passing style):

```
1 lengthCPS :: [a] -> Int
2 lengthCPS = length' id
3   where length' :: (Int -> Int) -> [a] -> Int
4         length' f [] = f 0
5         length' f (_:xs) = length' (\lxs -> f (1 + lxs)) xs
```

Chvostová rekúzia pomocou CPS

Predošlá implementácia síce využíva chvostovú rekúziu, ale nie je o nič efektívnejšia ako **lengthR**:

- `length' id [1,2,3]`
- `length' ( $\lambda xs \rightarrow id (1 + xs)$ ) [2,3]`
- `length' ( $\lambda xs \rightarrow (\lambda xs \rightarrow id (1 + xs)) (1 + xs)$ ) [3]`
- `length' ( $\lambda xs \rightarrow (\lambda xs \rightarrow (\lambda xs \rightarrow id (1 + xs)) (1 + xs)) (1 + xs)$ ) []`
- `( $\lambda xs \rightarrow (\lambda xs \rightarrow (\lambda xs \rightarrow id (1 + xs)) (1 + xs)) (1 + xs)$ ) 0`
- `( $\lambda xs \rightarrow (\lambda xs \rightarrow id (1 + xs)) (1 + xs)$ ) (1 + 0)`
- `( $\lambda xs \rightarrow (\lambda xs \rightarrow id (1 + xs)) (1 + xs)$ ) 1`
- `( $\lambda xs \rightarrow id (1 + xs)$ ) (1 + 1)`
- `( $\lambda xs \rightarrow id (1 + xs)$ ) 2`
- `id (1 + 2)`
- `id 3`
- `3`

Chvostová rekúzia pomocou CPS

Predošlá implementácia síce využíva chvostovú rekúziu, ale nie je o nič efektívnejšia ako **lengthR**:

- `length' id [1,2,3]`
- `length' ( $\lambda lxs \rightarrow id (1 + lxs)$ ) [2,3]`
- `length' ( $\lambda lxs \rightarrow (\lambda lxs \rightarrow id (1 + lxs)) (1 + lxs)$ ) [3]`
- `length' ( $\lambda lxs \rightarrow (\lambda lxs \rightarrow (\lambda lxs \rightarrow id (1 + lxs)) (1 + lxs)) (1 + lxs)$ ) []`
- `( $\lambda lxs \rightarrow (\lambda lxs \rightarrow (\lambda lxs \rightarrow id (1 + lxs)) (1 + lxs)) (1 + lxs)$ ) 0`
- `( $\lambda lxs \rightarrow (\lambda lxs \rightarrow id (1 + lxs)) (1 + lxs)$ ) (1 + 0)`
- `( $\lambda lxs \rightarrow (\lambda lxs \rightarrow id (1 + lxs)) (1 + lxs)$ ) 1`
- `( $\lambda lxs \rightarrow id (1 + lxs)$ ) (1 + 1)`
- `( $\lambda lxs \rightarrow id (1 + lxs)$ ) 2`
- `id (1 + 2)`
- `id 3`
- `3`

Chvostová rekúzia pomocou CPS

Predošlá implementácia síce využíva chvostovú rekúziu, ale nie je o nič efektívnejšia ako **lengthR**:

- $\text{length}' \text{ id } [1,2,3]$
- $\text{length}' (\lambda lxs \rightarrow \text{id } (1 + lxs)) [2,3]$
- $\text{length}' (\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) [3]$
- $\text{length}' (\lambda lxs \rightarrow (\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) (1 + lxs)) []$
- $(\lambda lxs \rightarrow (\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) (1 + lxs)) 0$
- $(\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) (1 + 0)$
- $(\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) 1$
- $(\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + 1)$
- $(\lambda lxs \rightarrow \text{id } (1 + lxs)) 2$
- $\text{id } (1 + 2)$
- $\text{id } 3$
- 3

Chvostová rekúzia pomocou CPS

Predošlá implementácia síce využíva chvostovú rekúziu, ale nie je o nič efektívnejšia ako **lengthR**:

- `length' id [1,2,3]`
- `length' ( $\lambda lxs \rightarrow id (1 + lxs)$ ) [2,3]`
- `length' ( $\lambda lxs \rightarrow (\lambda lxs \rightarrow id (1 + lxs)) (1 + lxs)$ ) [3]`
- `length' ( $\lambda lxs \rightarrow (\lambda lxs \rightarrow (\lambda lxs \rightarrow id (1 + lxs)) (1 + lxs)) (1 + lxs)$ ) []`
- `( $\lambda lxs \rightarrow (\lambda lxs \rightarrow (\lambda lxs \rightarrow id (1 + lxs)) (1 + lxs)) (1 + lxs)$ ) 0`
- `( $\lambda lxs \rightarrow (\lambda lxs \rightarrow id (1 + lxs)) (1 + lxs)$ ) (1 + 0)`
- `( $\lambda lxs \rightarrow (\lambda lxs \rightarrow id (1 + lxs)) (1 + lxs)$ ) 1`
- `( $\lambda lxs \rightarrow id (1 + lxs)$ ) (1 + 1)`
- `( $\lambda lxs \rightarrow id (1 + lxs)$ ) 2`
- `id (1 + 2)`
- `id 3`
- `3`

Chvostová rekúzia pomocou CPS

Predošlá implementácia síce využíva chvostovú rekúziu, ale nie je o nič efektívnejšia ako **lengthR**:

- $\text{length}' \text{ id } [1,2,3]$
- $\text{length}' (\lambda lxs \rightarrow \text{id } (1 + lxs)) [2,3]$
- $\text{length}' (\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) [3]$
- $\text{length}' (\lambda lxs \rightarrow (\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) (1 + lxs)) []$
- $(\lambda lxs \rightarrow (\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) (1 + lxs)) 0$
- $(\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) (1 + 0)$
- $(\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) 1$
- $(\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + 1)$
- $(\lambda lxs \rightarrow \text{id } (1 + lxs)) 2$
- $\text{id } (1 + 2)$
- $\text{id } 3$
- 3

Chvostová rekúzia pomocou CPS

Predošlá implementácia síce využíva chvostovú rekúziu, ale nie je o nič efektívnejšia ako **lengthR**:

- $\text{length}' \text{ id } [1,2,3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) [2,3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) [3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + \text{lxs})) []$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + \text{lxs})) 0$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + 0)$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) 1$
- $(\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + 1)$
- $(\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) 2$
- $\text{id } (1 + 2)$
- $\text{id } 3$
- 3

Chvostová rekúzia pomocou CPS

Predošlá implementácia síce využíva chvostovú rekúziu, ale nie je o nič efektívnejšia ako **lengthR**:

- $\text{length}' \text{ id } [1,2,3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) [2,3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) [3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + \text{lxs})) []$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + \text{lxs})) 0$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + 0)$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) 1$
- $(\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + 1)$
- $(\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) 2$
- $\text{id } (1 + 2)$
- $\text{id } 3$
- 3

Chvostová rekúzia pomocou CPS

Predošlá implementácia síce využíva chvostovú rekúziu, ale nie je o nič efektívnejšia ako **lengthR**:

- $\text{length}' \text{ id } [1,2,3]$
- $\text{length}' (\lambda lxs \rightarrow \text{id } (1 + lxs)) [2,3]$
- $\text{length}' (\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) [3]$
- $\text{length}' (\lambda lxs \rightarrow (\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) (1 + lxs)) []$
- $(\lambda lxs \rightarrow (\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) (1 + lxs)) 0$
- $(\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) (1 + 0)$
- $(\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) 1$
- $(\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + 1)$
- $(\lambda lxs \rightarrow \text{id } (1 + lxs)) 2$
- $\text{id } (1 + 2)$
- $\text{id } 3$
- 3

Chvostová rekúzia pomocou CPS

Predošlá implementácia síce využíva chvostovú rekúziu, ale nie je o nič efektívnejšia ako **lengthR**:

- $\text{length}' \text{ id } [1,2,3]$
- $\text{length}' (\lambda lxs \rightarrow \text{id } (1 + lxs)) [2,3]$
- $\text{length}' (\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) [3]$
- $\text{length}' (\lambda lxs \rightarrow (\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) (1 + lxs)) []$
- $(\lambda lxs \rightarrow (\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) (1 + lxs)) 0$
- $(\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) (1 + 0)$
- $(\lambda lxs \rightarrow (\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + lxs)) 1$
- $(\lambda lxs \rightarrow \text{id } (1 + lxs)) (1 + 1)$
- $(\lambda lxs \rightarrow \text{id } (1 + lxs)) 2$
- $\text{id } (1 + 2)$
- $\text{id } 3$
- 3

Chvostová rekúzia pomocou CPS

Predošlá implementácia síce využíva chvostovú rekúziu, ale nie je o nič efektívnejšia ako **lengthR**:

- $\text{length}' \text{ id } [1,2,3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) [2,3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) [3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + \text{lxs})) []$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + \text{lxs})) 0$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + 0)$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) 1$
- $(\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + 1)$
- $(\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) 2$
- $\text{id } (1 + 2)$
- $\text{id } 3$
- 3

Chvostová rekúzia pomocou CPS

Predošlá implementácia síce využíva chvostovú rekúziu, ale nie je o nič efektívnejšia ako **lengthR**:

- $\text{length}' \text{ id } [1,2,3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) [2,3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) [3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + \text{lxs})) []$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + \text{lxs})) 0$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + 0)$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) 1$
- $(\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + 1)$
- $(\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) 2$
- $\text{id } (1 + 2)$
- $\text{id } 3$
- 3

Chvostová rekúzia pomocou CPS

Predošlá implementácia síce využíva chvostovú rekúziu, ale nie je o nič efektívnejšia ako **lengthR**:

- $\text{length}' \text{ id } [1,2,3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) [2,3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) [3]$
- $\text{length}' (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + \text{lxs})) []$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + \text{lxs})) 0$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) (1 + 0)$
- $(\lambda \text{ lxs} \rightarrow (\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + \text{lxs})) 1$
- $(\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) (1 + 1)$
- $(\lambda \text{ lxs} \rightarrow \text{id } (1 + \text{lxs})) 2$
- $\text{id } (1 + 2)$
- $\text{id } 3$
- 3

Chvostová rekúzia s akumulátormi

Konverzia nechvostovej rekúzie na chvostovú rekúziu môže vyžadovať:

- viac akumulátorov
- akumulátor s komplexnejšou štruktúrou

```
1 fibTR :: Integer -> Integer
2 fibTR = fib' 0 1 where
3   fib' f1 _ 0 = f1
4   fib' f1 f2 n = fib' f2 (f1+f2) (n-1)
```

reverse

nechvostová rekurzia:

```
1 reverse :: [a] -> [a]
2 reverse [] = []
3 reverse (x:xs) = reverse xs ++ [x]
```

chvostová rekurzia a akumulátorom:

```
1 reverse :: [a] -> [a]
2 reverse = rev' [] where
3     rev' a [] = a
4     rev' a (x:xs) = rev' (x:a) xs
```

append

nechvostová rekúzia:

```
1 append :: [a] -> [a] -> [a]
2 append [] x = x
3 append x [] = x -- nemusí byť
4 append (x:xs) y = x : append xs y
```

chvostová rekúzia s akumulátorom:

```
1 append :: [a] -> [a] -> [a]
2 append x y = app' y (reverse x)
3   where
4     app' a [] = a
5     app' a (x:xs) = app' (x:a) xs
```

Nepovinná domáca úloha

Úloha

Čo robí nasledovná `divna` funkcia?

```
1 divna = \f g -> (. f g)
```

Pôvodne vznikla chybou pri pokuse o zápis `\f g -> ((.) f g)`.

Riešenie

```
1 --           f           x           g           y
2 divna :: (a -> b -> c) -> a -> (c -> d) -> b -> d
3 divna f x g y = g (f x y)
```

Príklad

- `h x y = divna (+) x odd y`
- $h(x, y) = \text{True} \iff (x + y) = 1 \pmod{2}$

Nepovinná domáca úloha

Úloha

Čo robí nasledovná `divna` funkcia?

```
1 divna = \f g -> (. f g)
```

Pôvodne vznikla chybou pri pokuse o zápis `\f g -> ((.) f g)`.

Riešenie

```
1 --           f           x           g           y
2 divna :: (a -> b -> c) -> a -> (c -> d) -> b -> d
3 divna f x g y = g (f x y)
```

Príklad

- `h x y = divna (+) x odd y`
- $h(x, y) = \text{True} \iff (x + y) = 1 \pmod{2}$